

UTS PRAKTIKUM

IDENTITAS MAHASISWA

Keterangan	Isi
Nama	FARRELINO PUTRA SETIAWAN
NIM	362558302013
Kelas	2E TRPL
Tanggal Praktikum	5 Oktober 2026
Mata Kuliah	INTEROPERABILITAS
Judul Proyek	Pengembangan REST API Toko / <i>Merchandise</i> Menggunakan Laravel

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era pengembangan perangkat lunak modern, kebutuhan akan integrasi antar sistem (interoperabilitas) sangat tinggi. REST API (*Representational State Transfer Application Programming Interface*) menjadi standar arsitektur yang paling banyak digunakan karena kemampuannya menjembatani komunikasi data antar platform secara ringan menggunakan format JSON. Praktikum ini bertujuan untuk membangun, menguji, dan mendokumentasikan backend REST API mandiri untuk sistem manajemen toko *merchandise*.

1.2 Tujuan

- Menerapkan konsep arsitektur MVC (*Model-View-Controller*) pada framework Laravel.
- Mengimplementasikan operasi CRUD (*Create, Read, Update, Delete*) melalui *endpoint* API.
- Melakukan pengujian fungsionalitas menggunakan perangkat pengujian API (*Thunder Client*).
- Menyimpan kode sumber secara aman menggunakan *Version Control System* (Git/GitHub).

BAB II

ARSITEKTUR & STRUKTUR PROYEK

Proyek ini dikembangkan di lingkungan lokal menggunakan server ServBay dan framework Laravel dengan struktur komponen utama sebagai berikut:

1. **Routes (routes/api.php):** Berfungsi sebagai peta jalur untuk mendaftarkan seluruh *endpoint* API dengan awalan prefiks `/api`.
2. **Controller (App\Http\Controllers\ProductController):** Mengatur logika bisnis untuk merespons permintaan klien, memproses data, dan mengembalikan format JSON.
3. **Model (App\Models\Product):** Berperan sebagai jembatan komunikasi dengan basis data MySQL menggunakan Eloquent ORM.

Daftar Endpoint API:

Method	Endpoint	Deskripsi / Fungsi
GET	/api/products	Menampilkan seluruh daftar produk
POST	/api/products	Menambahkan data produk baru
PUT / PATCH	/api/products/{id}	Mengubah atau memperbaiki data produk berdasarkan ID
DELETE	/api/products/{id}	Menghapus data produk berdasarkan ID

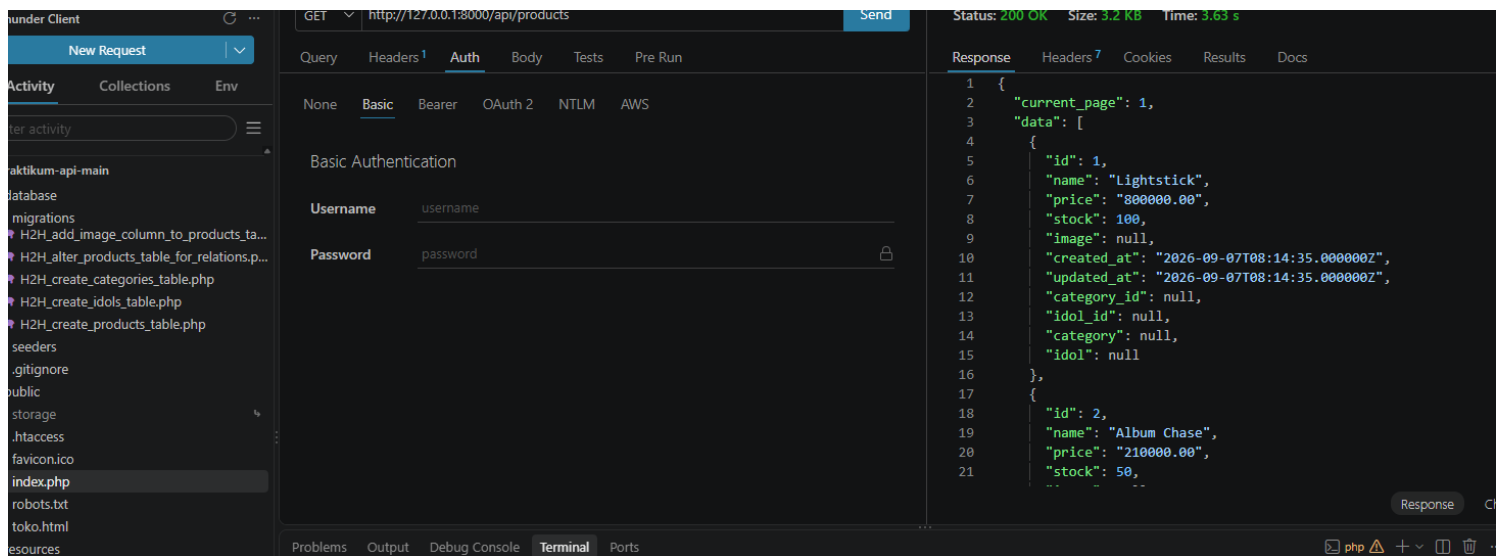
BAB III

HASIL PENGUJIAN API (THUNDER CLIENT)

Pengujian fungsionalitas dilakukan menggunakan ekstensi **Thunder Client** pada Visual Studio Code untuk memastikan seluruh *endpoint* mengembalikan *HTTP Status Code* dan struktur respons JSON yang benar.

3.1 Pengujian Method GET (/api/products)

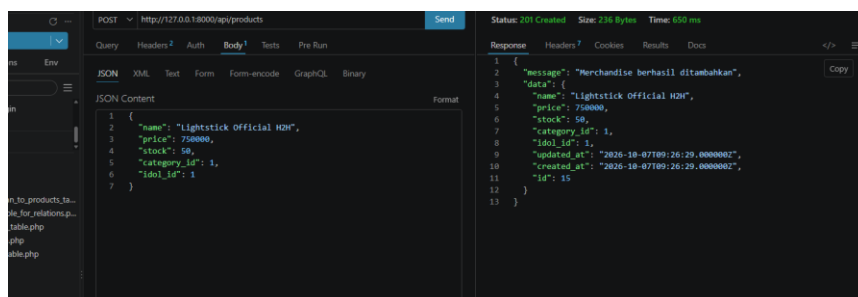
- **Deskripsi:** Mengirim permintaan untuk mengambil seluruh data produk yang tersimpan di dalam server lokal.
- **Status Code:** 200 OK (Berhasil)
- **Hasil Tangkapan Layar (Screenshot):**



Catatan visual pada pengujian: Menampilkan daftar produk seperti "Lightstick" dan "Album Chase" dengan format data JSON terstruktur.

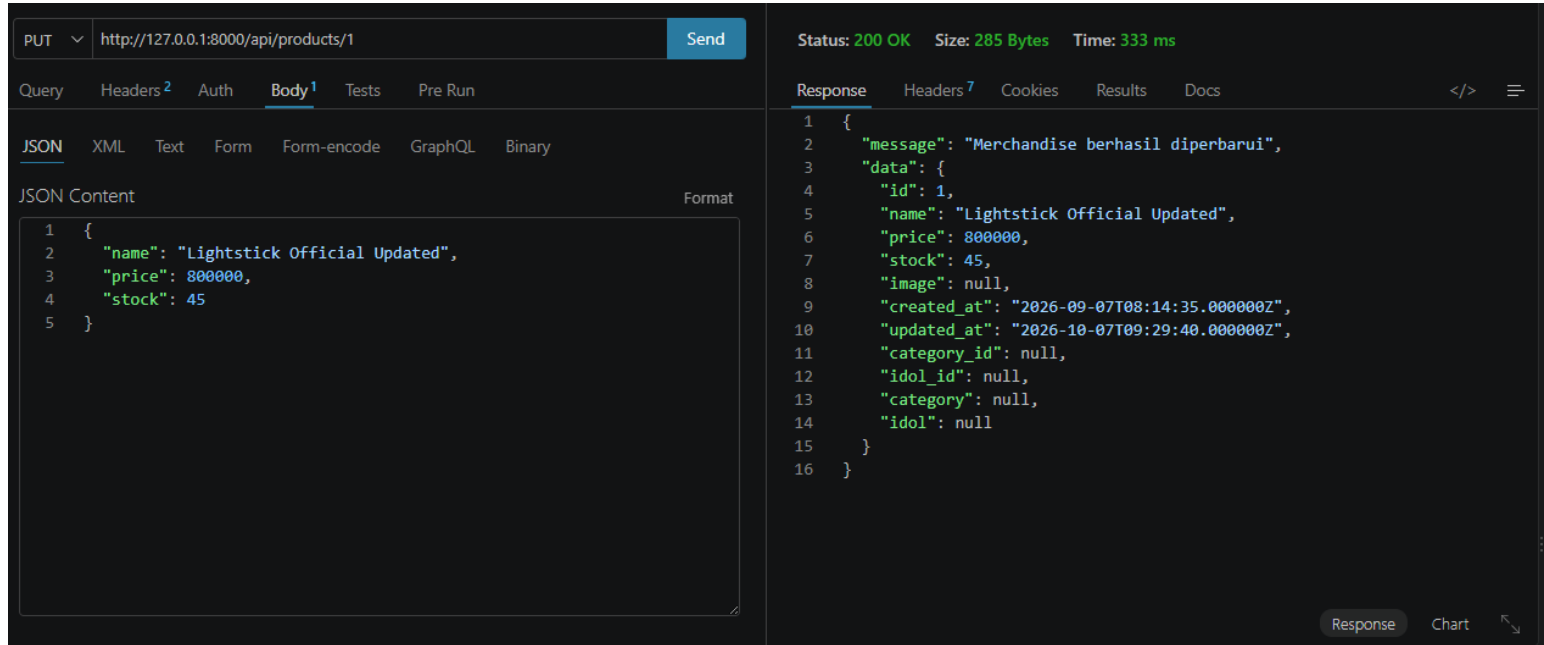
3.2 Pengujian Method POST (/api/products)

- **Deskripsi:** Mengirim data baru ke server untuk disimpan ke dalam *database*.
- **Status Code:** 201 Created (Berhasil menambahkan data)
- **Hasil Tangkapan Layar (Screenshot):**



3.3 Pengujian Method PUT / PATCH (/api/products/{id})

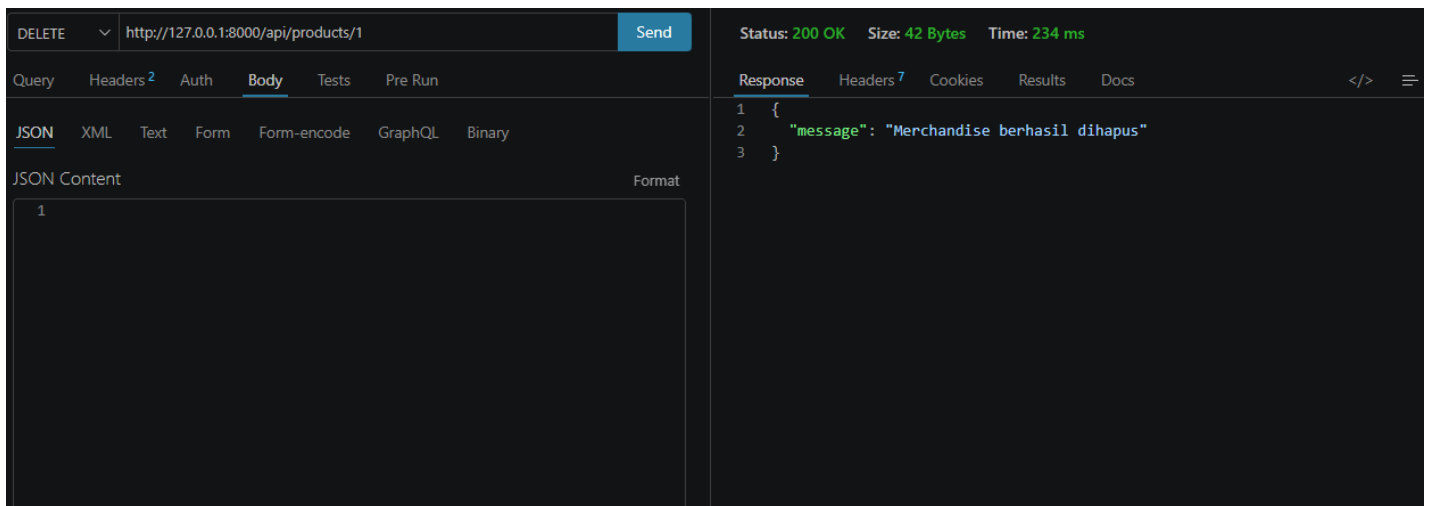
- **Deskripsi:** Mengubah atau memperbarui informasi data produk tertentu berdasarkan ID.
- **Status Code:** 200 OK (Berhasil memperbarui data)
- **Hasil Tangkapan Layar (Screenshot):**



- Status code menunjukkan **200 OK**.
- Bagian *Response* memperlihatkan perubahan data produk dengan ID tersebut.

3.4 Pengujian Method DELETE (/api/products/{id})

- **Deskripsi:** Menghapus data produk spesifik dari *database* berdasarkan ID.
- **Status Code:** 200 OK / 204 No Content (Berhasil menghapus data)
- **Hasil Tangkapan Layar (Screenshot):**



BAB IV

DOKUMENTASI API (OPENAPI / SWAGGER)

Dokumentasi sistem REST API dikembangkan menggunakan spesifikasi standar OpenAPI/Swagger untuk memudahkan pengembang lain dalam memahami cara kerja dan spesifikasi *endpoint* yang tersedia.

- **Deskripsi Dokumentasi:** Dokumentasi memuat informasi lengkap mencakup metode HTTP, URL *endpoint*, parameter yang diperlukan, format *request body*, struktur *response*, hingga standar *HTTP status code* yang digunakan.
- **Hasil Swagger / OpenAPI:**
- Berikut adalah dokumentasi spesifikasi *endpoint* REST API toko *merchandise* yang merangkum seluruh rute, parameter, format data, hingga *HTTP status code* sesuai standar OpenAPI:

No	Method	URL Endpoint	Parameter / Request Body	Struktur Response & Keterangan	HTTP Status Code
1	GET	/api/products	None (Tanpa parameter)	Menampilkan daftar seluruh data produk dalam bentuk array JSON.	200 OK
2	GET	/api/products/{id}	id (Path Parameter)	Menampilkan detail data produk berdasarkan ID tertentu.	200 OK, 404 Not Found
3	POST	/api/products	name (string, wajib) price (numeric, wajib) stock (integer, wajib)	Menyimpan data produk baru ke database dan mengembalikan pesan sukses.	201 Created, 400 Bad Request

No	Method	URL Endpoint	Parameter / Request Body	Struktur Response & Keterangan	HTTP Status Code
			category_id (integer) idol_id (integer)		
4	PUT	/api/products/{id}	id (Path Parameter) name, price, stock (JSON Body)	Memperbarui informasi data produk yang sudah ada berdasarkan ID.	200 OK, 404 Not Found
5	DELETE	/api/products/{id}	id (Path Paramet		

BAB V

JAWABAN PERTANYAAN ANALISIS

1. Mengapa REST API menggunakan JSON sebagai format pertukaran data?

Karena format JSON sangat ringan, mudah dibaca oleh manusia maupun mesin (*human and machine-readable*), serta didukung secara natif oleh hampir semua bahasa pemrograman dan platform modern (seperti web dan aplikasi *mobile*).

2. Apa perbedaan fungsi GET, POST, PUT, dan DELETE pada project ini?

- GET digunakan untuk mengambil atau membaca data dari server.
- POST digunakan untuk mengirim dan menambahkan data baru ke dalam *database*.
- PUT / PATCH digunakan untuk mengubah atau memperbarui data yang sudah ada berdasarkan ID tertentu.
- DELETE digunakan untuk menghapus data spesifik dari *database*.

3. Mengapa response API perlu menggunakan HTTP status code yang sesuai?

Agar aplikasi *client* (seperti *frontend* atau aplikasi *mobile*) dapat mengetahui dengan pasti status atau hasil dari permintaan yang dikirimkan (misalnya kode 200 untuk sukses, 201 untuk data berhasil dibuat, 400 untuk kesalahan *request*, atau 404 jika data tidak ditemukan).

4. Apa yang terjadi jika dua aplikasi berbeda menggunakan API yang sama?

Sistem akan berjalan secara *interoperable*, di mana satu *backend* API yang sama dapat melayani berbagai platform berbeda secara bersamaan (misalnya aplikasi berbasis web dan aplikasi *mobile* dapat mengakses sumber data produk yang sama secara serentak).

5. Jelaskan bagaimana database MySQL, REST API, dan aplikasi client saling berkomunikasi.

Aplikasi *client* mengirimkan permintaan HTTP (*request*) ke REST API $\rightarrow$ REST API memproses logika bisnis dan berinteraksi dengan *database* MySQL melalui Eloquent Model $\rightarrow$ MySQL mengembalikan data ke API $\rightarrow$ API merespons balik ke *client* dalam bentuk format JSON.

6. Jika API akan digunakan oleh aplikasi mobile, perubahan apa yang perlu diperhatikan dari sisi keamanan?

Perlu diperhatikan penggunaan autentikasi berbasis *token* yang aman (seperti Laravel Sanctum atau JWT), penerapan enkripsi data via HTTPS, pencegahan celah keamanan seperti *SQL Injection*, serta pembatasan jumlah akses (*rate limiting*) untuk menghindari serangan *brute force*.

BAB VI

KESIMPULAN

Proyek Ujian praktikum REST API toko *merchandise* ini telah berhasil dikembangkan secara utuh menggunakan Laravel. Seluruh rute, pengontrol, dan basis data saling terintegrasi dengan baik sehingga mampu melakukan operasi CRUD secara optimal melalui format pertukaran data JSON. Pengujian via Thunder Client serta dokumentasi OpenAPI/Swagger juga membuktikan bahwa sistem merespons permintaan dengan kode status HTTP yang akurat dan siap diintegrasikan.

4.1 Lampiran (Source Code Repository)

Seluruh kode program (*source code*) proyek ini dapat diakses secara publik melalui repositori GitHub pada tautan berikut:

Repository URL: <https://github.com/linodefontaine/praktikum-api-main.git>